IEOR 4574E001 · AI ENGINEERING APPS

COLUMBIA UNIVERSITY

LESSON 02

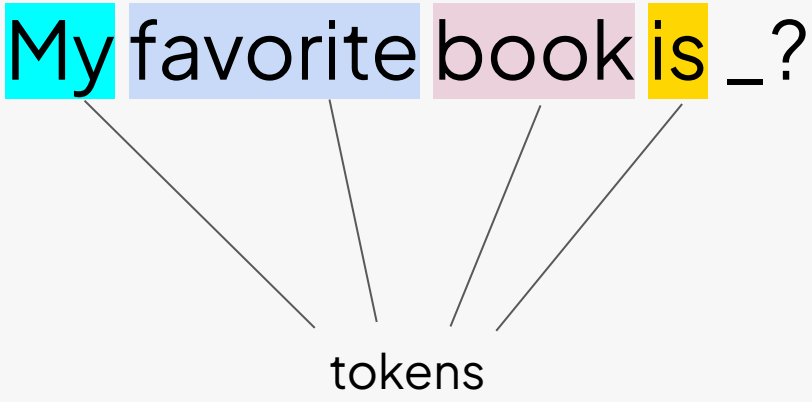
Introduction to LLMs and LLMOps

02

Ciro Greco

Columbia University · 2026

Language models predict sequences



Moby Dick	4.1%
War and Peace	3.9%
the one	1.4%
written	2.2%

Pick on of these, but with a bit of randomness

argmax ---------- **sampling**

temperature

COLUMBIA UNIVERSITY

Architectures to predict sequences

n-grams → learn from counts. Models estimate probability from counts; simple and fast, but suffer from sparse statistics as n grows, and can't model long contexts or generalize beyond observed n-grams.

RNNs (2014) → learn from sequences. Recurrent nets replace fixed n with a hidden state, letting the model in principle use unbounded context. Practically, they struggle with vanishing/exploding gradients and long-range dependencies.

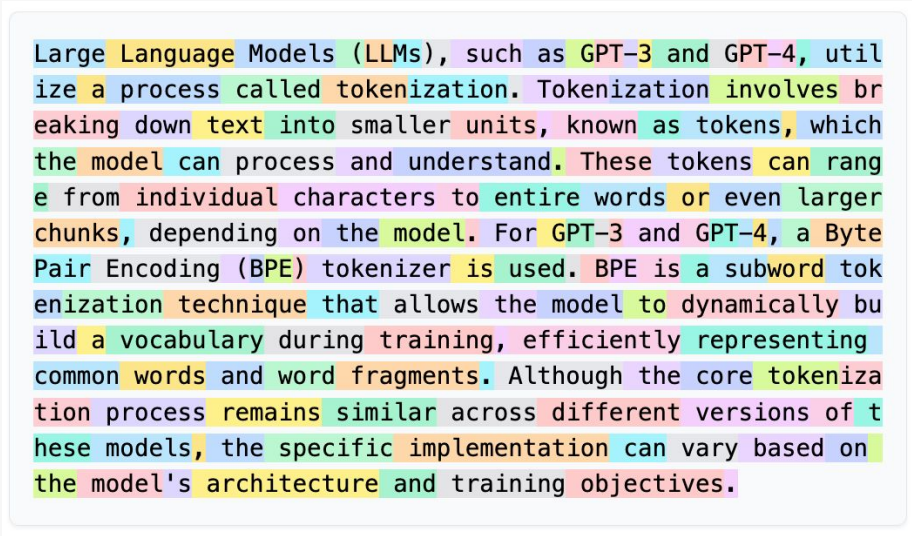
LSTMs → mitigate vanishing gradients. LSTMs add gates and persistent cell states, making RNN training more stable and improving long-range patterns. They power early state-of-the-art seq2seq systems, but are still sequential.

Attention (2015) → break the "thought-vector" bottleneck. Vanilla seq2seq compresses an input into one vector; attention lets the decoder soft-search over all encoder states each step.

Transformers (2017) → scale with self-attention. Self-attention replaces recurrence with direct token-to-token interactions; multi-head attention captures multiple relationships; positional encodings restore order; and the architecture trains in parallel over tokens.

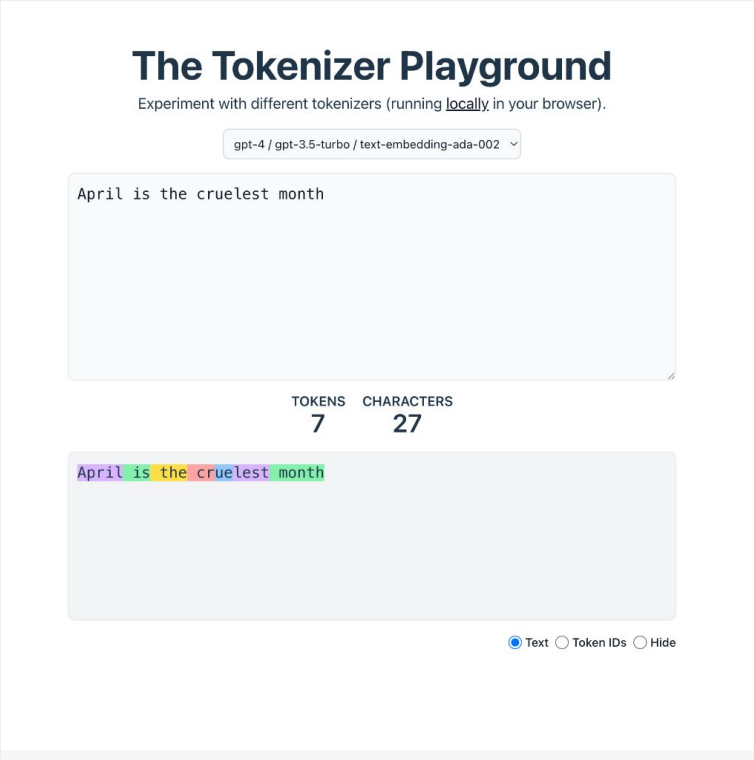
Tokenization

- **Tokens.** Language models predict next items in sequences: tokens are the items to be predicted.
- **Tokenizer.** A mapping from text strings to ids tied to a model's vocabulary.
- **Tokenization strategies**. Bytes, characters, words, subwords tokenizers.
- The tokenization strategy impact directly inference cost and latency.



Tokenization strategies

Two sentences that look the same to you may differ by a factor of two in tokens once encoded.





COLUMBIA UNIVERSITY

Tokenization strategies

The same sequence may differ by a factor of two in tokens once encoded depending on the tokenizer.

Method	Training principle	Example output	Used in
BPE	Merge most frequent symbol pairs	`["un", "believ", "able", " stories"]`	GPT-2, LLaMA
WordPiece	Maximize corpus likelihood; mark continuations with ##	`["un", "##believ", "##able", "stories"]`	BERT
SentencePiece	Train on raw text (spaces = token "_"); optionally unigram LM	`["_un", "believable", "_stories"]`	T5, XLNet

COLUMBIA UNIVERSITY

Tokenization matters a lot in practice

Money

Priced per token. Input and output at different rates.

Limits

The context window is a token count, not a word count.

Speed

Time-to-first-token can grow with the square of input tokens.

Not portable

Tokenizers are per-model. No token count survives a model or a vendor change.

Words as vectors: bag of words

Corpus:

The dog is happy. The child makes the dog happy. The dog makes the child happy

↓

Tokenization:

D1: [The] [dog] [is] [happy]
D2: [The] [child] [makes] [the] [dog] [happy]
D3: [The] [dog] [makes] [the] [child] [happy]

↓

Documents	Counting word frequencies
D1	the: 1, dog: 1, is: 1, happy: 1
D2	the: 2, dog: 1, makes: 1, child: 1, happy: 1
D3	the: 2, child: 1, makes: 1, dog: 1, happy: 1

↓

Encode	child	dog	happy	is	makes	the	BoW Vector representations
D1	0	1	1	1	0	1	[0,1,1,1,0,1]
D2	1	1	1	0	1	2	[1,1,1,0,1,2]
D3	1	1	1	0	1	2	[1,1,1,0,1,2]

Corpus
A collection of text document.

Tokenizer
Divide the text sequence into smaller units and map them onto ids.

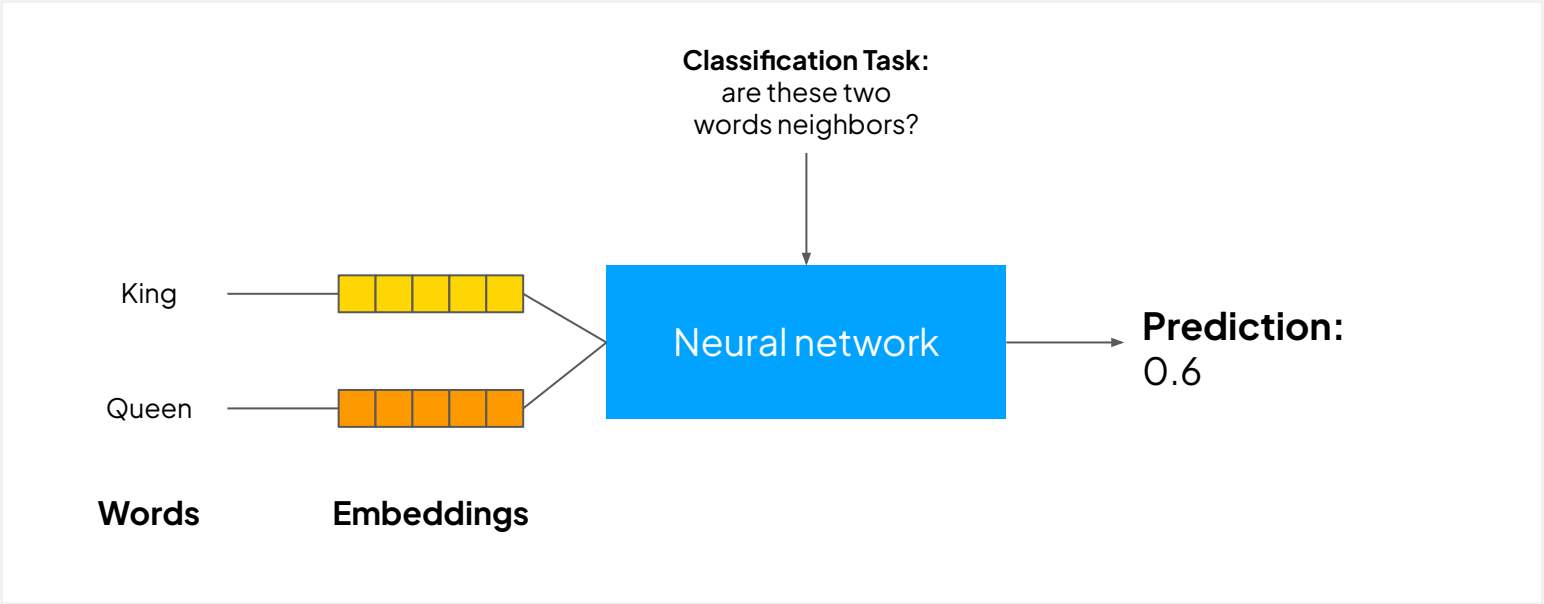
Word frequencies
A vocabulary of unique words and the number of times they occur in each document in the Corpus

Word encoder
Encode the text as a vector for each document. Each element of the vector represents the frequency of a particular word in the document.

COLUMBIA UNIVERSITY

Words as vectors: dense vector embeddings

- **Meaning and context:** dense vectors capture distributional similarity ("you shall know a word by the company it keeps").



Columbia University

Words as vectors: cosine similarity

- **Geometry encodes meaning:** use cosine similarity as a proxy for semantic relatedness.

$$\cos\theta = \frac{u \cdot v}{\|u\| \, \|v\|}$$

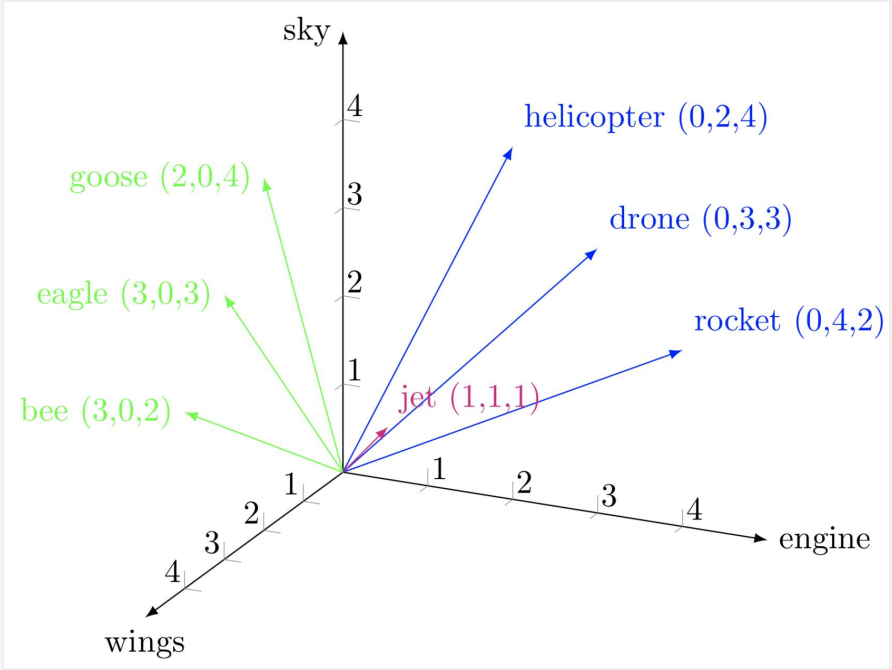


Image from Corpus Linguistics and Statistics with R
http://www.springer.com/gp/book/9783319645704

COLUMBIA UNIVERSITY

Training static embeddings: Word2Vec

Context prediction

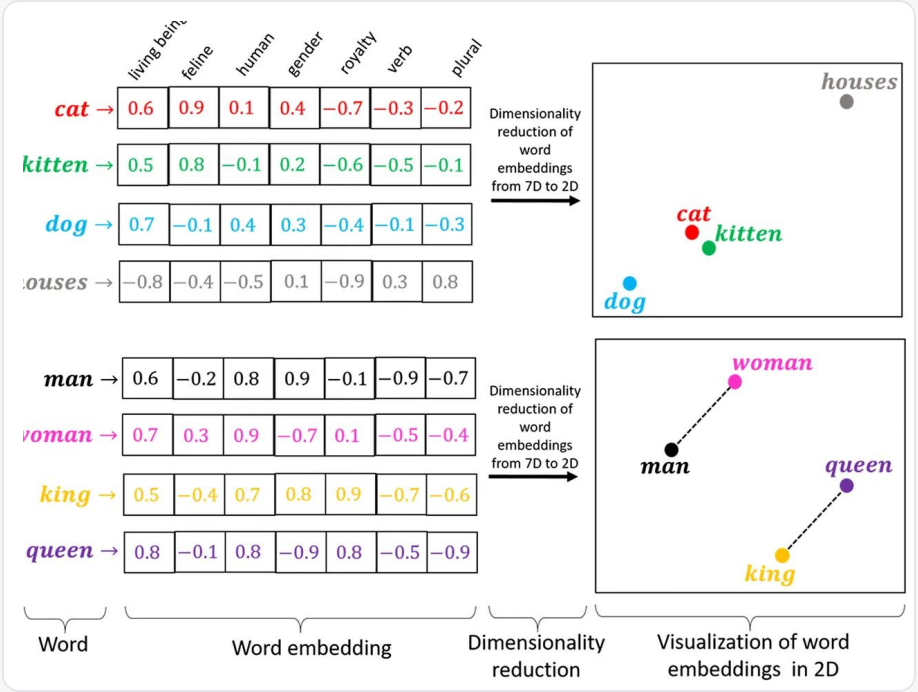
- **CBOW:** Predicts the centre word from context. Fast and reliable.
- **Skip-gram:** Predicts context from centre word. Slower but better for rare words.

Very scalable

- Labels are free via self-supervision.
- Vector distance/direction encode rich semantic relations.
- Pretrain once, reuse everywhere across downstream tasks.

One vector per word type

- A single representation for distinct meanings (e.g., ***"bank"*** as river vs. money).
- High intuition, but limited on polysemy.



COLUMBIA UNIVERSITY

Types of embeddings

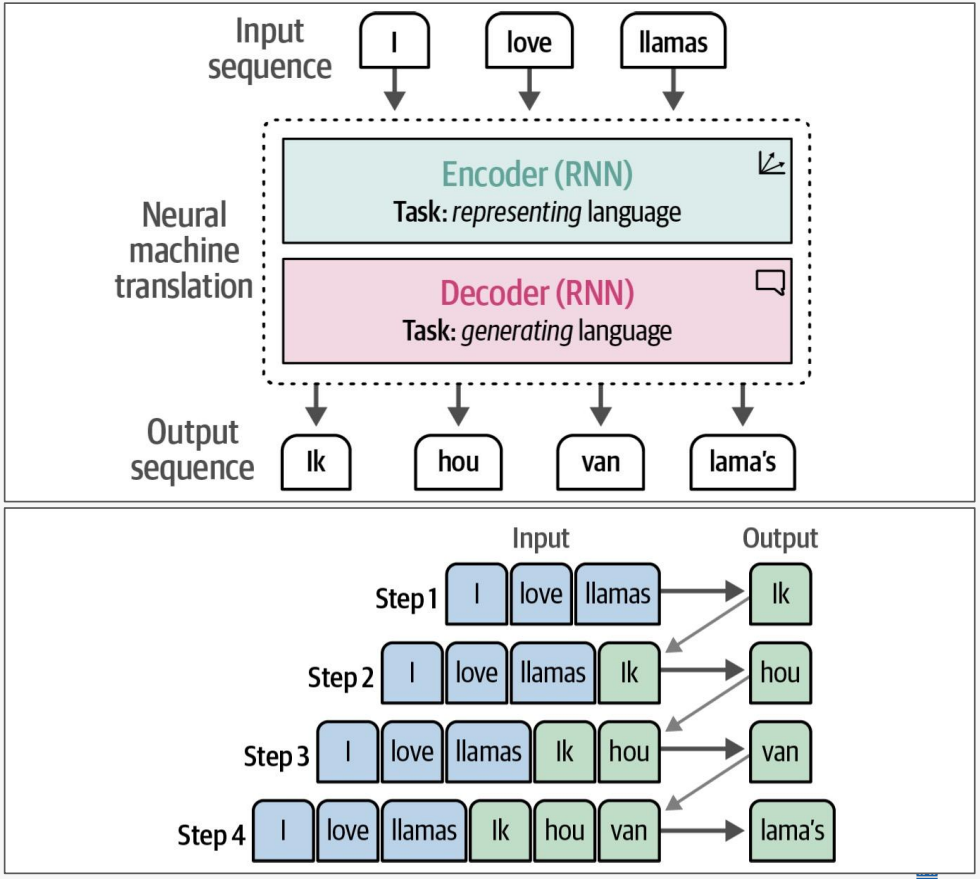
The word "embedding" names different things.

	ENCODES	WHY IT EXISTS
Token	what this word is	The model need to compute on vectors
Positional	where it sits in the sequence	We need to encode position in a sequence
Contextual	what it means here	One vector per word collapses polysemy
Sentence / Document	what the whole text is about	Search compares whole texts
Multimodal	what an image or clip contains	Text and images must share one space

The first three are stages of one forward pass. **The last two come from purpose-trained models.**

Encoder–decoder

- **Encoder**: reads the input sequence step by step and compresses it into a hidden representation (context vector).
- **Decoder**: takes this representation and generates the output sequence, one token at a time.
- **Training**: decoder is guided with the correct previous word (teacher forcing).
- **Inference**: decoder relies on its own predictions (autoregressive loop).



Images from Jay Alammar, Maarten Grootendorst

Context-thought bottleneck

- **Seq2seq compresses the whole input into one fixed-size vector:** it works on short, simple inputs BUT drops details on long, complex sentences.
- **Symptoms**: wrong word order, missing entities, brittle agreement across clauses.
- **Motivation for attention**: let the decoder look back at all encoder states, not just one summary.

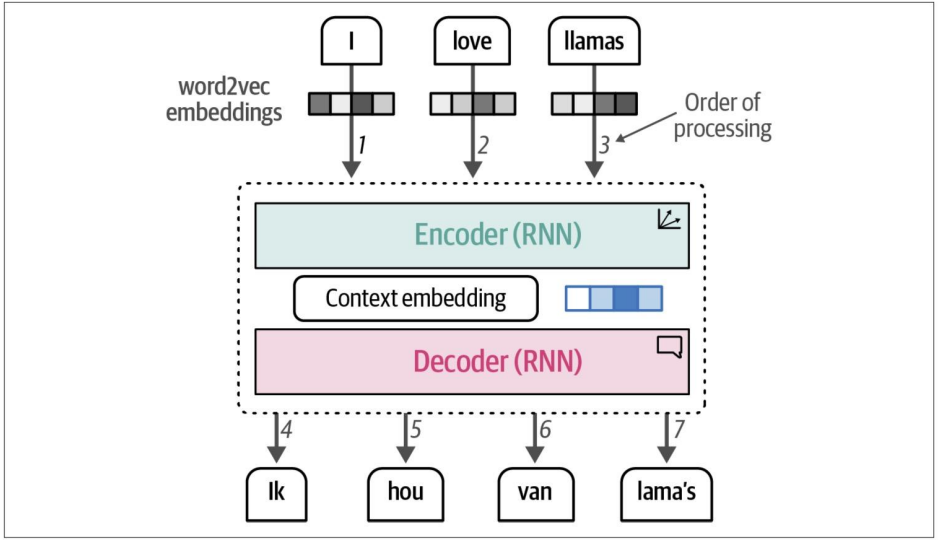
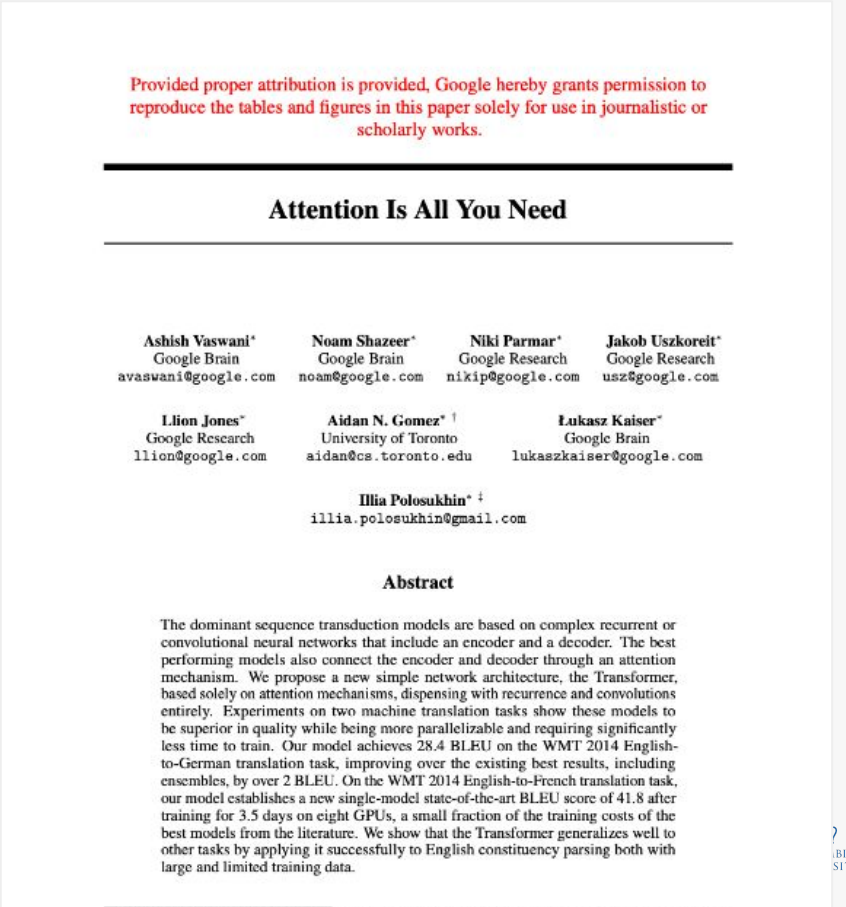


Figure 1-13. Using word2vec embeddings, a context embedding is generated that represents the entire sequence.

Images from Jay Alammar, Maarten Grootendorst

Attention is all you need

- In 2017, attention replaces recurrence in sequence models.
- **Key idea:** tokens attend to tokens directly (no more RNN loop).
- Attention unlocks parallel training, longer-range dependencies, clean scaling.
- This architecture enabled **BERT, GPT, T5** and the foundation model era.



Provided proper attribution is provided, Google hereby grants permission to reproduce the tables and figures in this paper solely for use in journalistic or scholarly works.

Attention Is All You Need

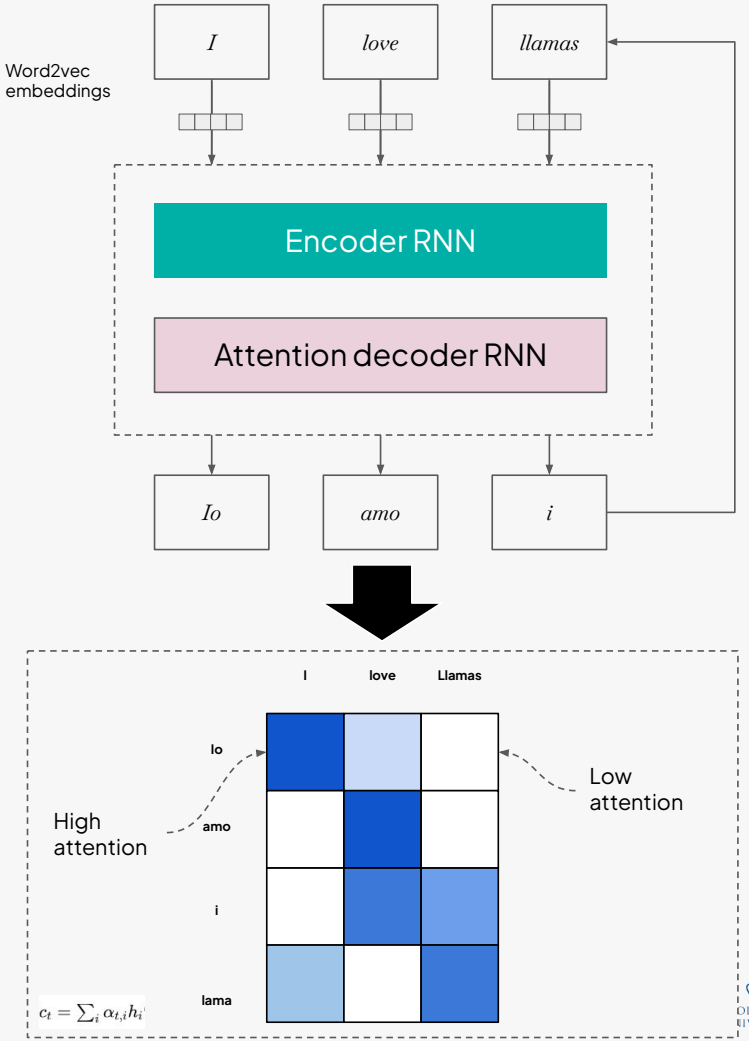
Ashish Vaswani[*] Google Brain avaswani@google.com
Noam Shazeer[*] Google Brain noam@google.com
Niki Parmar[*] Google Research nikip@google.com
Jakob Uszkoreit[*] Google Research usz@google.com
Llion Jones[*] Google Research llion@google.com
Aidan N. Gomez[*] [†] University of Toronto aidan@cs.toronto.edu
Łukasz Kaiser[*] Google Brain lukaszkaiser@google.com
Illia Polosukhin[*] [‡] illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

Attention in seq2seq

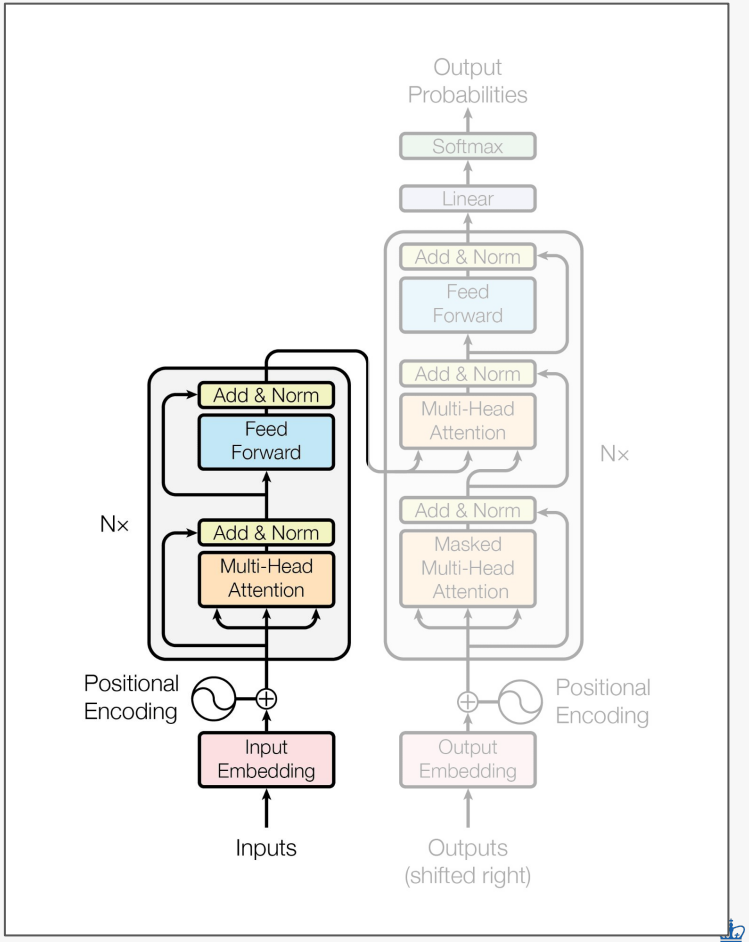
- Attention fixes the problem of one single “thought vector” dropping details on long inputs.
- The decoder now computes a weighted look-back over all encoder states.
- Benefit: soft alignments, better long sentences, partial interpretability.



Transformer encoder

- Builds global, order-aware token states (not one summary), fixing long-range context.
- Each word looks at the whole sentence so nothing important is forgotten.

- Input embeddings + positional info.
- Multi-head self-attention (bidirectional).
- Feed-forward network (per token).
- Residual connections and LayerNorm.
- **Output:** contextual token representations.

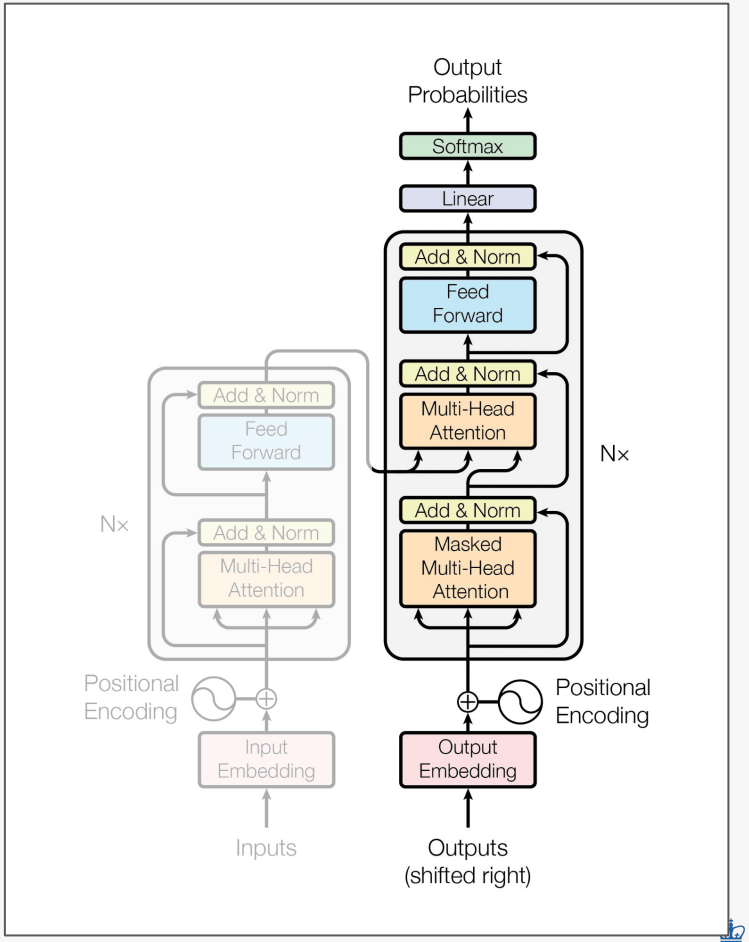


COLUMBIA UNIVERSITY

Transformer decoder

- Generates causally while retrieving source info via cross-attention for faithful outputs.
- It writes the next word and checks the original input so the answer stays on topic.

- Masked self-attention over generated tokens.
- Cross-attention to encoder outputs.
- Feed-forward network.
- Residuals and LayerNorm.
- **Output:** next-token logits for autoregressive generation.

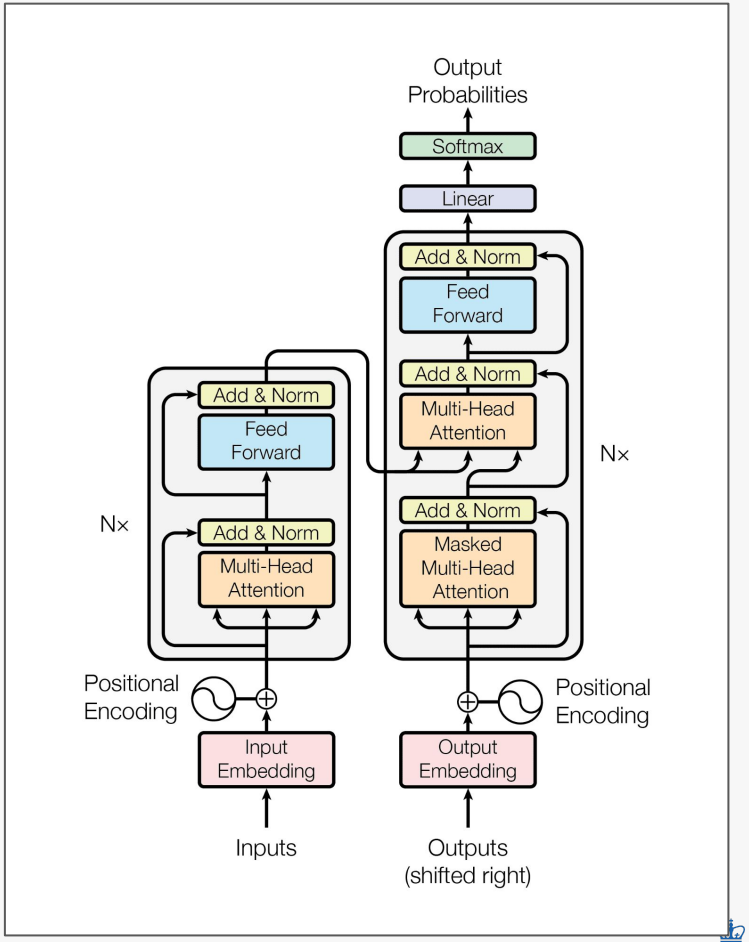


COLUMBIA UNIVERSITY

Putting it all together

- Encoder builds contextual representations; decoder generates with masked self-attention and cross-attention, enabling parallel training and strong long-range modeling.

- Encoder stack produces source representations.
- Decoder stack generates targets using masked self-attention + cross-attention.
- Parallelizable training, strong long-range modeling.
- Families: encoder-only (BERT), decoder-only (GPT), encoder-decoder (T5).



COLUMBIA UNIVERSITY

Using LLMs

Running and calling models

	SELF-HOSTED	PROVIDER API
What you get	a folder of files	a URL and a key
Who owns the GPU	you	them
What you can change	all of it	the fields in the request
What you pay for	the machine, by the hour	tokens, by the million
Why you would pick it	the data cannot leave, or scale makes it cheaper	everything else, and where you start

Running and calling models

	YOU DECIDE	SELF-HOSTED	PROVIDER API
1	**which model**	`the folder you downloaded`	`the model field`
2	**what it reads**	`the string you build`	`systemInstruction + contents`
3	**how it picks each next token**	`generation_config.json`	`temperature, topP`
4	**when it stops, and in what shape**	`your serving code`	`maxOutputTokens, responseSchema`
5	**what you learn from the call**	`your own logging`	`finishReason, usage`

Anatomy of an API call

```
POST …/gemini-2.5-flash-002:generateContent
{
 "model": "gemini-2.5-flash-002",
 "systemInstruction": {"parts":[{"text":"<returns policy>"}]},
 "contents": [
     {"role": "user",  "parts":[{"text":"<first message>"}]},
     {"role": "model", "parts":[{"text":"<our reply>"}]},
     {"role": "user",  "parts":[{"text":"<latest message>"}]}
   ],
 "generationConfig": {
     "temperature": 0.2,
     "topP": 0.95,
     "maxOutputTokens": 512,
     "responseSchema": {"type": "object", …}
   }
}
```

1 Which model

Pinned to a dated build. A floating alias changes behaviour on a morning you shipped nothing.

2 What it reads

The policy plus the whole conversation, re-sent every turn. This is your token bill.

3 How it picks each token

Low temperature for a task with one right answer.

4 When it stops, and the shape

A cap you should always set, and a schema that returns an object instead of prose.

Anatomy of an API call

```
{
  "candidates": [{
     "content": {"parts":[{"text":"<the answer>"}]},
     "finishReason": "STOP"
   }],
  "usageMetadata": {
     "promptTokenCount": 2841,
     "cachedContentTokenCount": 2560,
     "candidatesTokenCount": 96,
     "totalTokenCount": 2937
   }
}
```

FinishReason

STOP, MAX_TOKENS or SAFETY. Three different events that produce text looking exactly the same.

UsageMetadata

The only ground truth about what a call cost. 2,841 tokens in, 96 out. The answer is the short part.

Example of self-hosted model

meta-llama/Llama-3.1-8B-Instruct

`model-0000{1..4}.safetensors`	**1**	the weights, 16.1 GB in four shards
`config.json`	**1**	layers, heads, dtype, context length
`tokenizer.json`	**1**	what counts as a token, 128,256 of them
`tokenizer_config.json`	**2**	the chat template: messages become one string
`generation_config.json`	**3**	temperature 0.6, topP 0.9, three stop tokens
`you write this part`	**4** **5**	stopping, output shape, and all your logging

Pre-training

Training phases

Phase	Purpose	Data	Outcome
Pretraining	Learn general linguistic and world knowledge	Large unlabeled text corpus	Base LLM
Instruction Tuning	Learn to follow instructions/tasks	Supervised human-labeled data	Instruct model
Preference Optimization	Align with human values/preferences	Human feedback, rankings	Chat/assistant model
Domain/Modality Adaptation	Specialize or extend capabilities	Domain or multimodal data	Specialized model

COLUMBIA UNIVERSITY

Self-supervision unlocks scalability

- No manual labels: the data is its own target (next/masked token).
- Each sentence yields thousands of training pairs → labels '**for free**'.

Input: "The cat sat on the **[MASK]"**

Label: "mat" (comes directly from the original sentence).

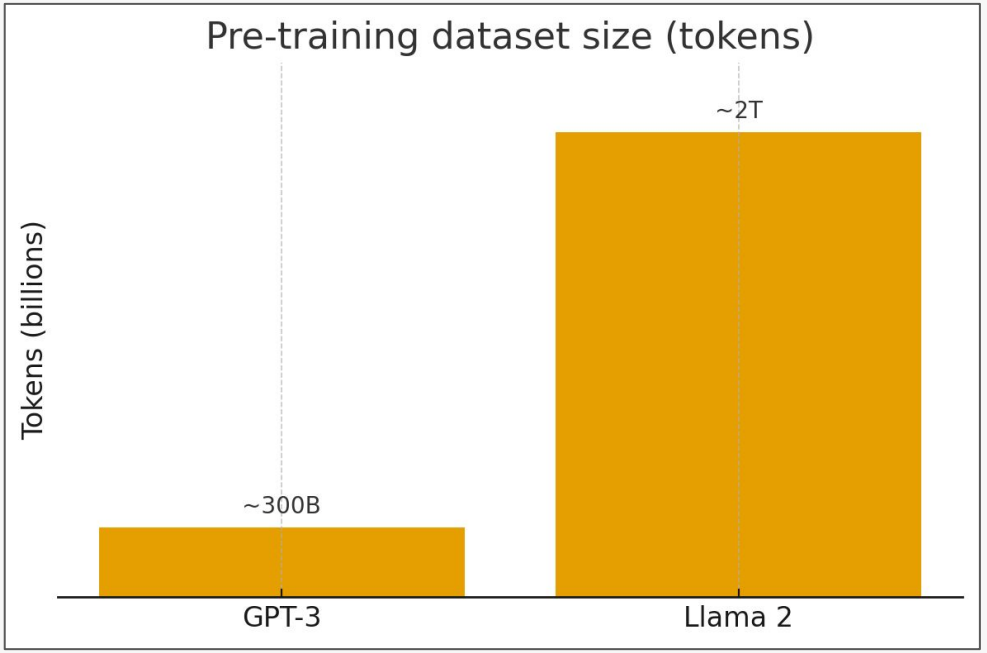
<BOS>	the
<BOS>, the,	cat
<BOS>, the, cat	sat
<BOS>, the, cat, sat	on
<BOS>, the, cat, sat, on	the
<BOS>, the, cat, sat, on, the	mat

Training set

COLUMBIA UNIVERSITY

Self-supervision unlocks scalability

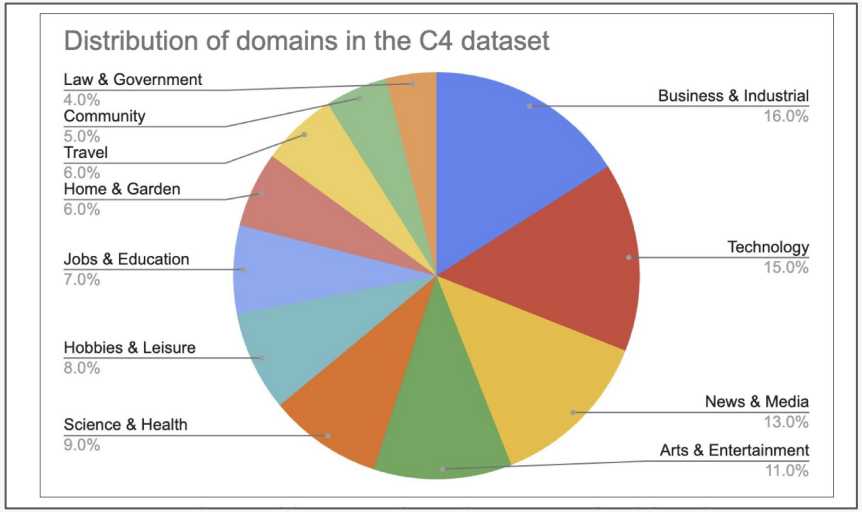
- Self-supervision allows for dataset sizes that are simply **infeasible to label by hand.**
- Remember the scaling laws paper by Kaplan et al. (2020).
- No manual labeling but filtering, deduplicating, domain weighting, synthetic data generation.



Columbia University

Pre-training

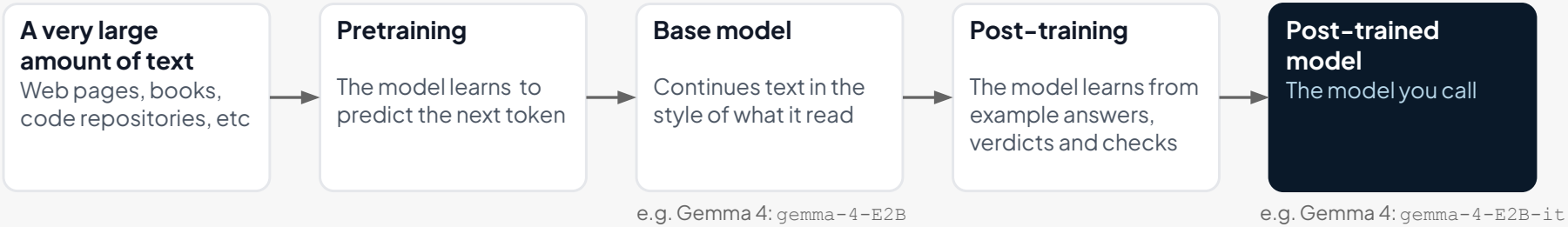
- **Objective:** predict missing/next token (masked vs. autoregressive).
- **Scale:** no human labels → web-scale corpora.
- **Data recipe:** diverse, multi-domain (and increasingly multilingual).
- **Cost reality:** pre-training dominates total compute.
- **Output:** a powerful base model (not yet aligned).



COLUMBIA UNIVERSITY

Post-training

Pretraining and post-training



- **Pretraining** produces the base model and its knowledge in the weights.
- **Post-training** produced the behaviours an application relies on.
  - Adjust the weights using three kinds of material:
    - example answers to imitate,
    - verdicts from people or other models on which of two answers is better,
    - code that checks whether an answer is right, such as tests that pass.

Post-trained models

- Open models publish both versions.
  - The chat and agent models from a provider's API are typically post-trained.
  - Base models are published by open-model families, for those who run their own post-training.

- Post-training decides whether a model adheres to your format, calls your tools, follows your system instruction and refuses what it should.



Behaviours for your application

Many important features of an LLM application come from behaviors that are post-trained into the model

TRAINED BEHAVIOUR	WHAT YOU BUILD ON IT
Following an instruction in a set format	Extraction, classification, summaries your code can parse
Calling tools	Agents that look up records and take actions
Thinking before answering	Multi-step reasoning, maths, code
Giving the system instruction priority over the user	Assistants that keep to a policy when users push back
Deciding when to refuse	Any public-facing product

Post-training signals

A training signal is the information that tells training which outputs to make more likely.
While pretraining relies on the next token in the dataset, post-training signals vary based on design goals:

SFT Supervised Fine Tuning.

Demonstration

Example answers to imitate.

RLHF Reinforcement Learning from Human Feedback.

Preference

A judge's verdict that one answer is better than another.
The judge is a person or a model.

RLVR Reinforcement learning with verifiable rewards

Automated check

Code verifies an answer, such as tests that pass providing a reward.

DISTILLATION

Teacher's output

A student model is trained to behave like a teacher model, usually a larger one, or like several teachers at once.

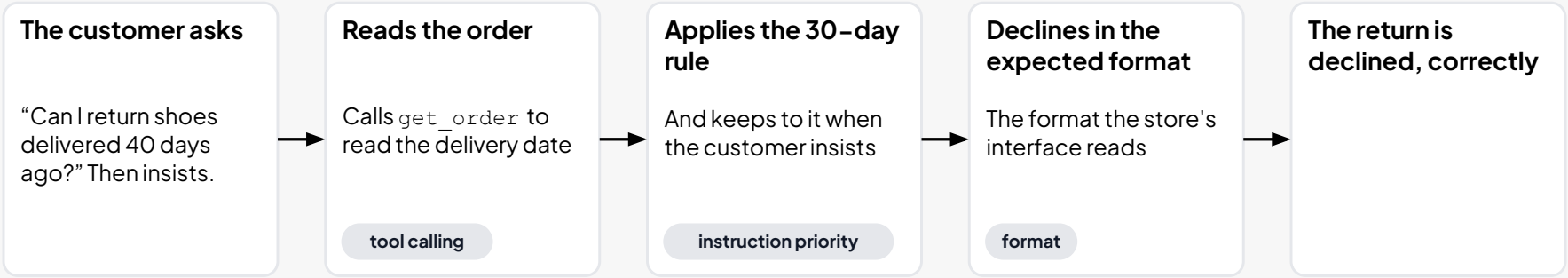
Signals may co-exist in you application

Can I return shoes that were delivered 40 days ago?

Where it comes from	*Signal*	*Method*	
01 People write ideal answers	**Demonstration** The ideal reply: "Those shoes arrived 40 days ago, outside the 30-day window."	**SFT** Imitate the answer	**The model you call**
02 A person or a model compares answers	**Preference** That reply is better than "Yes, you can return them."	**RLHF** Make the preferred answer more likely	
03 Code checks the result	**Check** The order database shows no refund: pass. A polite reply that refunded fails.	**RLVR** The check's result is the reward	
04 A larger model answers	**Teacher's output** The large model's reply, and how likely it found each next token.	**Distillation** A smaller model learns to match it	

Evals, evals, evals

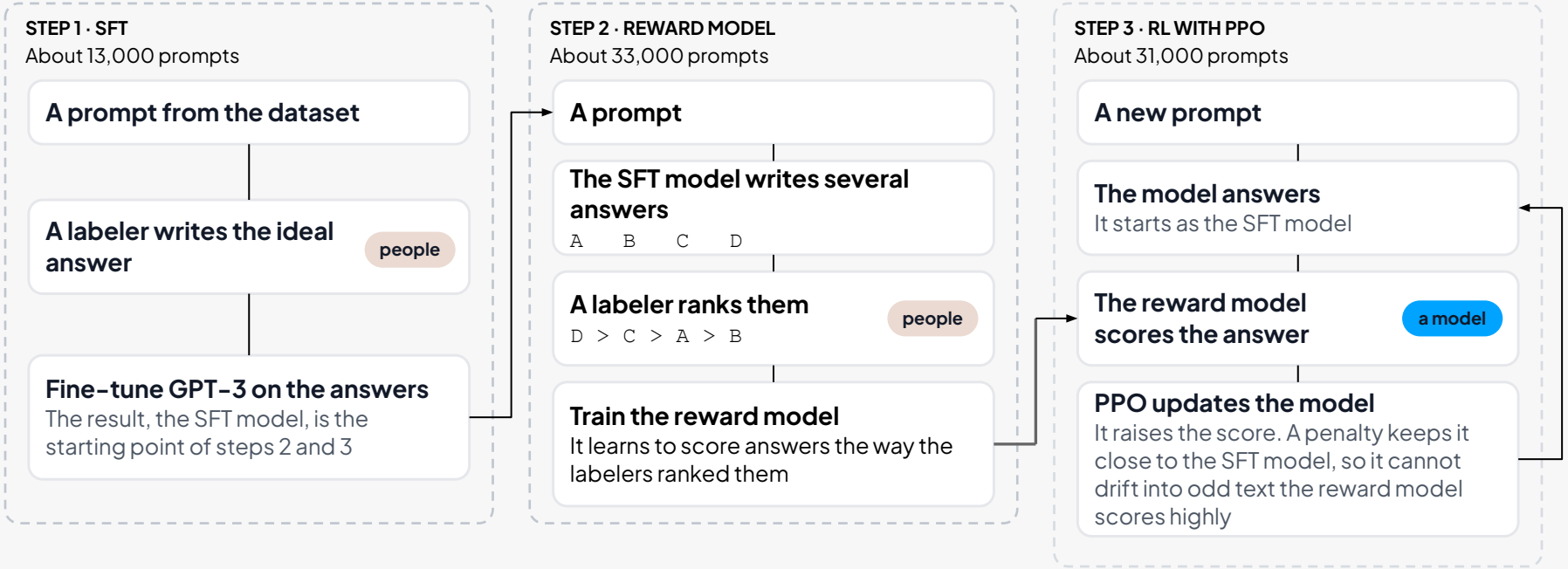
A store's customer-service agent, with a 30-day return rule in its system instruction



- You application uses several trained behaviours at once.
- If any one of these behaviours is weak, the application will fail, and nothing in your code shows why.
- Test every behaviour your application depends on with an *eval*, a set of test inputs with a check on each output, on the model you use.

SFT + RLHF

InstructGPT, OpenAI's post-trained GPT-3 (2022). [Ouyang et al. 2022]



COLUMBIA UNIVERSITY

Post-training is changing constantly: 2023 vs. 2026

2022 · INSTRUCTGPT

SFT
People write example answers (about 13,000 prompts)

people

→

Reward model + PPO
People rank answers. RL trains against a model of the rankings

people

2024 TO 2026 · OPEN RECIPES

SFT
Data mostly written by models: over 98% for Nemotron-4 340B

a model

→

Preference training
Models judge the pairs. DPO replaces the reward model and PPO

a model

→

RL against checks
Code checks the result. DeepSeek-V3.2: 1,827 simulated environments

code

→

Combine specialists
One student learns from domain specialists: more than ten for DeepSeek-V4

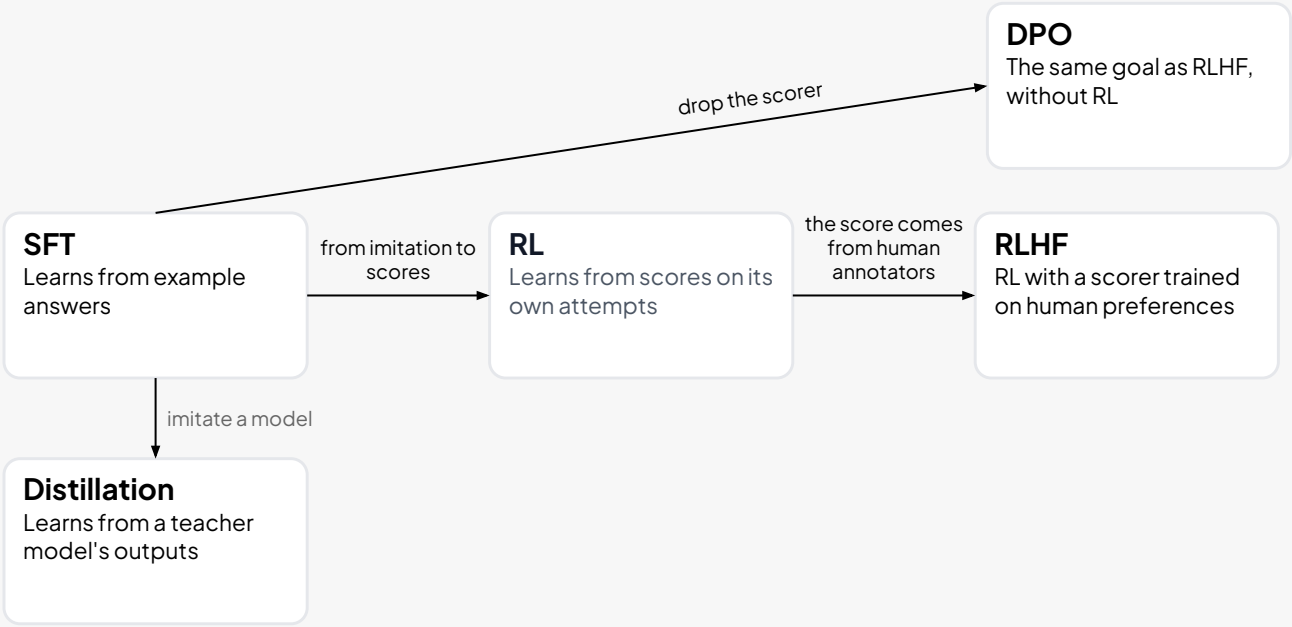
a model

Post-training techniques



COLUMBIA UNIVERSITY

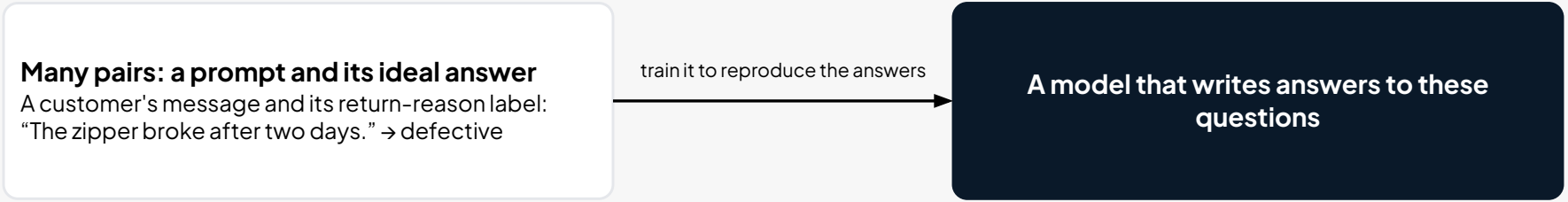
Post-training techniques



LoRA
An optimization technique to run any of them an a minimal set of weights: keep the model frozen and train only a small adapter.

SFT: learn from examples of the answer you want

- Supervised fine-tuning (SFT) shows the model examples of the answer you want and trains the model to write them.
- It is the same next-token training as pretraining, on curated pairs of a prompt and an ideal answer.
- Think of it as learning by copying worked examples: the model sees the right answer and learns to produce answers like it.
- What it teaches: a format, a tone, a task, and when to stop. Every published post-training recipe starts with it.



SFT: how it works

01 Collect pairs
People write them, existing datasets are converted, or a stronger model writes them. Start with people.

02 Format them
Each pair is written in a chat template: special tokens that mark who is speaking and where each turn ends.

03 Mask the prompt
Only the answer's tokens count in the loss, so the model learns to write answers and not to predict the user prompt.

04 Train
Training raises the probability of each answer token. The end-of-turn token is part of the answer, so the model learns when to stop.

05 Keep the best checkpoint
The saved weights that score best on examples held out of training.

`<|im_start|>user The zipper broke.<|im_end|>`	masked
`<|im_start|>assistant`	masked
`defective<|im_end|>`	**in the loss**

<|im_start|> and <|im_end|> mark each turn; <|im_end|> is the end-of-turn token.



Pick the right template!
A model run with a different chat template from the one it was trained with can perform drastically worse.

SFT in practice

WHERE IT IS USED

- The first stage of every published post-training recipe.
- Teams tune closed models on managed services (Google Agent Platform, Microsoft Foundry, AWS Bedrock; OpenAI only for existing customers since May 2026).
- Open models tune on Together, Fireworks, SageMaker AI or Tinker, or with Hugging Face's TRL library.

PROS

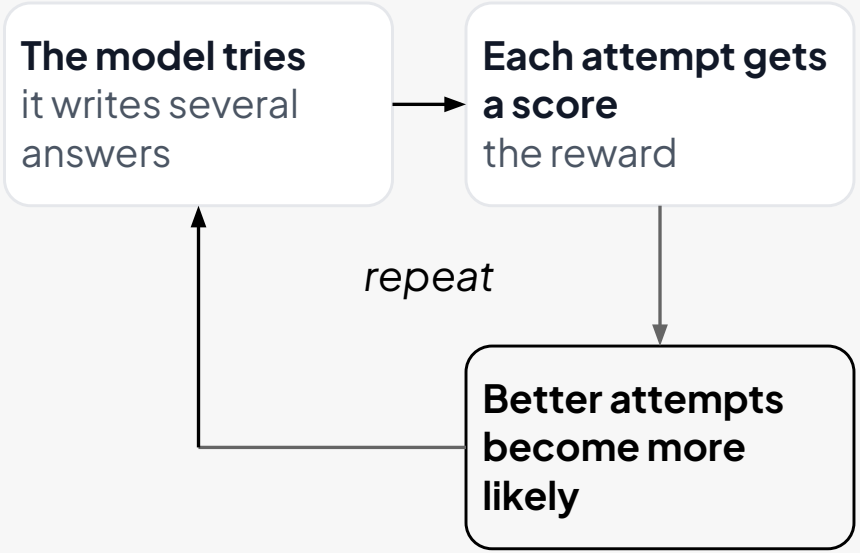
- Simple and stable.
- The most widely offered tuning method.
- Format, tone and task-following need little data.

CONS

- It only imitates: it never learns which answers are worse.
- Imitators copy style more readily than accuracy.
- It adds facts slowly, and facts learned this way raise hallucination.
- In published head-to-head tests on new facts, retrieval, looking up documents at request time, beat tuning.
- Narrow tuning can erase other abilities.

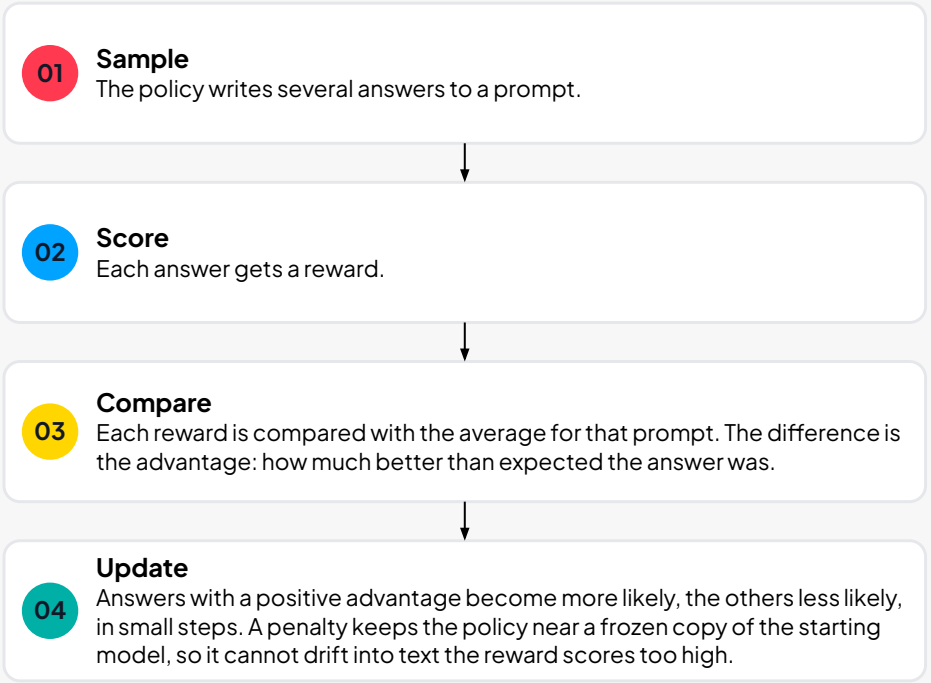
RL: learn from scores on your own attempts

- Reinforcement learning (RL) lets the model try, scores each attempt, and makes the better-scoring attempts more likely.
- The difference from SFT: SFT shows the model the answer. RL tells the model how good its own answer was.
- What it needs is a score for any answer, called the reward, and no written examples.
- It teaches skills where an answer can be scored, such as reasoning, maths, code and using tools.



RL: how it works

- RL repeats one loop.
- The model being trained is called the policy.
- The reward can come from:
  - people's preferences: RLHF
  - formal tests pass: RLVR
- Algorithms differ in how they get the average for the previous step. Examples:
  - PPO (2017) trains a value model to predict the reward an answer will get.
  - GRPO (2024) samples a group of answers to the same prompt and uses the group's average.
- Easy and impossible tasks teach nothing because if every answer scores the same, every advantage is zero and nothing changes.



RL in practice

WHERE IT IS USED

- Reasoning models: DeepSeek-R1–Zero, trained with RL and no SFT first, went from 15.6% to 77.9% on a maths competition with RL against checks alone.
- Agents trained in simulated environments with automatic checks.

PROS

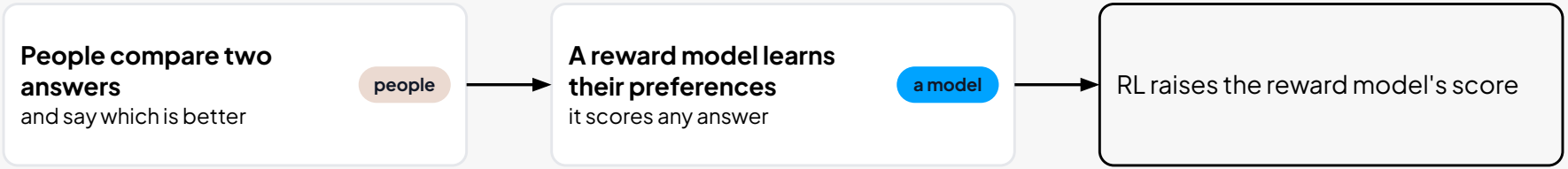
- Learns from its own attempts, without written examples.
- Improves skills that can be checked.
- Less forgetful than narrow SFT.

CONS

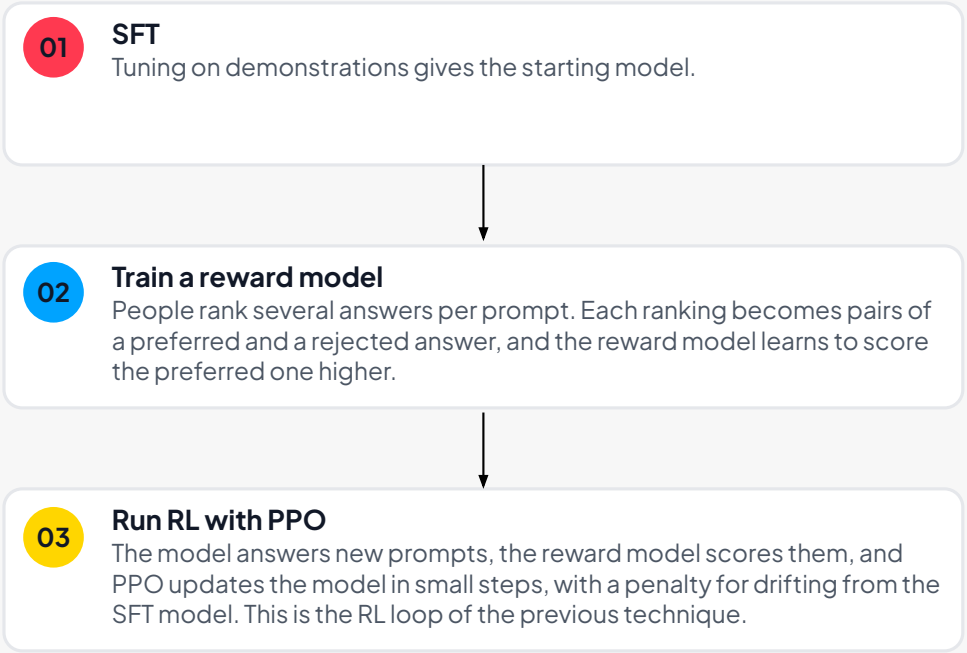
- The model optimizes the score, including gaps in the check: labs document reward hacking in their own runs, such as a model editing the tests it had to pass.
- Expensive, because the model writes text during training.

RLHF: RL with a score learned from human preferences

- Reinforcement learning from human feedback (RLHF) is RL whose score comes from a model trained to predict which of two answers people prefer.
  - **Why preferences.** People find it easier to compare two answers than to write the best one. Comparisons teach qualities that are hard to write down as one ideal answer, such as which of two helpful replies helps more.
  - It teaches helpfulness and caution of the assistants you call.



RLHF: how it works



- **Four models needed:**
  - The model being trained.
  - PPO's value model, which predicts the reward an answer will get.
  - The frozen starting model.
  - The reward model, which scores finished answers.
- **AI feedback.** Labs now replace many human labels with a model that judges answers against written principles (RLAIF, reinforcement learning from AI feedback), as in Anthropic's Constitutional AI.

RLHF in practice

WHERE IT IS USED

- Vendors say "RLHF" loosely for any training on preferences, and closed labs name no algorithm.
- The recipe behind ChatGPT's launch, and the source of much of the helpfulness, tone and refusals of today's assistants.

PROS

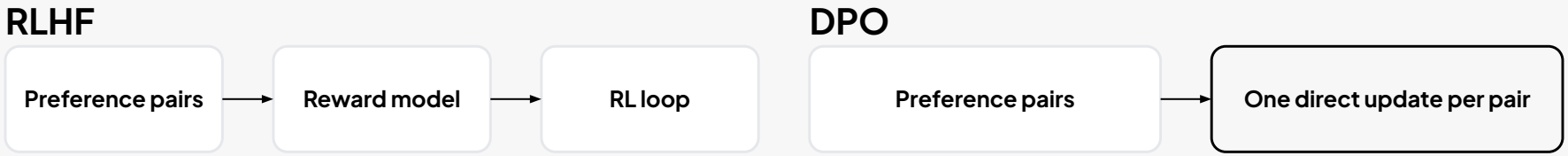
- Teaches fine grained qualities that people can judge but not easily write.
- Can push a bad answer down, which SFT cannot.

CONS

- Complex and expensive: several models, and text generated during training.
- The reward model is a proxy the model learns to exploit, and its biases ship as side effects: longer answers (verbosity), agreement with the user (sycophancy), overconfidence.
- Human labels are noisy: people disagree a lot.

DPO: preference training without RL

- Direct Preference Optimization (DPO) trains directly on pairs of answers: it raises the probability of the preferred one and lowers the rejected one. It aims at the same result as RLHF, with no reward model and no RL loop.
- No separate scorer to train, and no text generated during training. That makes it simpler and cheaper than RLHF.
- It keeps the goal: answers people prefer, without drifting far from the starting model.
- **How it works without a scorer.** The comparison with a frozen starting model acts as the reward: an answer the trained model makes more likely than the starting model counts as rewarded.



DPO: how it works

01 Collect pairs
A prompt with a chosen and a rejected answer, gathered before training.

02 Keep a reference
A frozen copy of the starting model, usually the SFT model, sits beside the model being trained.

03 Set β
A setting that controls how far the model may move from the reference: a larger β keeps it closer.

04 Update
For each answer, compare how likely the trained model makes it with how likely the reference makes it. Raise that ratio for the chosen answer, lower it for the rejected one.

The reference plays the part of RL's drift penalty: without it, the model could raise the chosen answer and drift too much from the base model.

DPO in practice

WHERE IT IS USED

- Wherever you can use RLHF.
- Actually, you should start from here because it is simpler.
- The usual preference stage of open recipes such as Tulu 3, Llama 3 and Olmo 3.

PROS

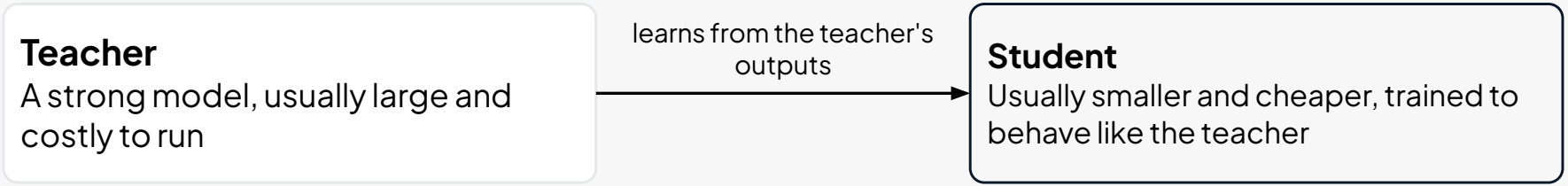
- Simple, stable and cheap.
- Pairs can come from your application, such as a user's edit against the draft or a judge's choice between two drafts.

CONS

- Offline: it learns only from pairs collected in advance, never from its own new mistakes.
- Quality peaks and then falls as training goes on, often within one pass over the data (Rafailov et al., 2024, DPO's authors).
- The pairs matter more than the algorithm, and they carry their biases, such as a preference for longer answers.

Distillation: a student learns from a teacher

- Distillation trains a student model to imitate a teacher model.
- The student is usually smaller and cheaper, so distillation moves a strong behaviour into a model that is faster to run.
- **Different from SFT.** SFT imitates example answers while distillation imitates a particular model.
- It teaches a large model's behaviour, in a small one.
- Distillation is one reason small models come close to large ones.



Distillation: how it works

Distillation comes in three forms, by what the student sees of the teacher.

01 **The teacher's text**
The teacher answers a set of inputs, and the student is trained on those answers with SFT.

02 **The teacher's probabilities**
The student matches the teacher's probabilities for every possible next token, which also say which wrong answers were plausible.

03 **The teacher's scores on the student's text**
The student writes, the teacher scores each token, and the student moves toward what the teacher finds likely. This is on-policy distillation: the student learns on the text it actually writes, mistakes included.

What a closed API allows. Closed APIs return text and at most the top 20 token probabilities of their own answers. Form 2 needs the probability of every possible next token, so distilling from a closed model is mostly the first form. The other two need a teacher whose probabilities you can get: an open-weight model, or a service that scores text you supply.

Distillation in practice

Distillation is how the small tiers you call are made, and a team's cheapest way to a small, fast model.

WHERE IT IS USED

- Small tiers: Google states its Gemini 2.5 models of "Flash size and below" are distilled.
- On the same small model, learning from DeepSeek-R1's outputs beat running RL on the small model directly.
- Teams: a large model labels data and a small model learns from it.

PROS

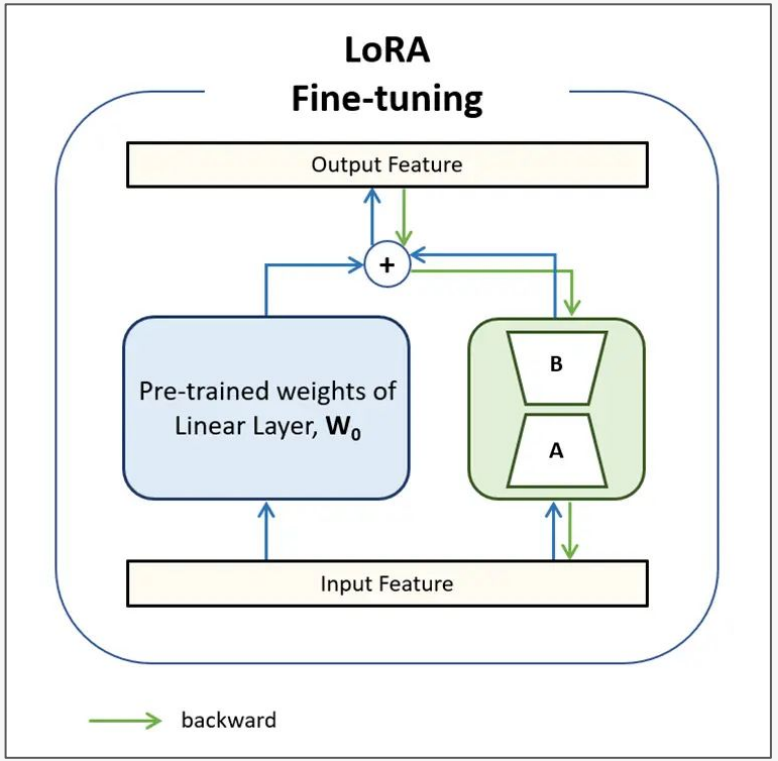
- Moves a strong behaviour into a small, fast, cheap model.
- Needs only unlabelled inputs and a teacher, plus a small human-labelled test set to check the result.

CONS

- The student inherits the teacher's errors, and weak teachers and labels that appear rarely in the data break it.
- Most API terms bar using outputs to build a competing model, with exceptions that differ by provider.
- Through a closed API the student sees only text.

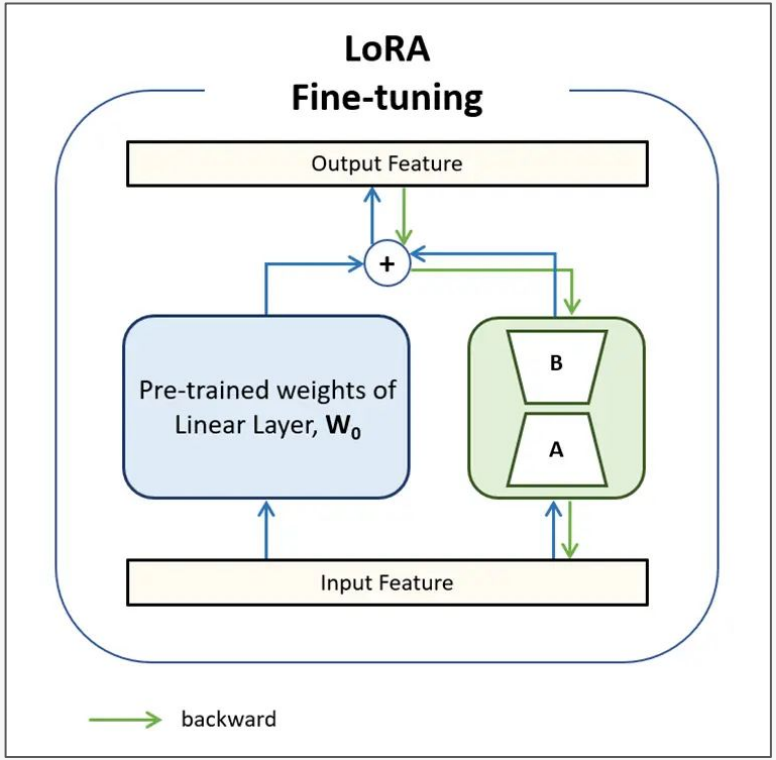
Low-Rank Adaptation (LoRa)

- **Low Rank representation.** A full 9 × 9 matrix W with 81 parameters can be approximated as the product of two matrices A and B with 9×1 and 1×9 parameters respectively.
- The total parameters of AB is 18 (not 81).
- **Key intuition**: if a matrix doesn't need all of its degrees of freedom to represent the change you want, you don't need to learn all of its parameters.



Low-Rank Adaptation (LoRa)

- LoRA is a PEFT technique that applies this insight systematically across the transformer.
- It does not add new layers and does not modify the original model weights directly.
- Instead, it learns **small low-rank matrices** that represent how the original weights should change for a new task and can be merged back to the original layers.
- No additional strain on inference cost and performance.



Columbia University

LoRA: how it works

01

Pick the matrices
Usually every linear layer, the weight matrices that make up most of the model.

02

Add two thin matrices
B and A sit beside each frozen matrix W, and their product, scaled by a setting called alpha, is added to W's output. The rank r, such as 8, sets their thickness: for a 4,096 × 4,096 matrix of 16.8 million weights, a rank-8 adapter trains about 65,000.

03

Start from the original model
B starts at zero, so the adapter adds nothing until training moves it. Only A and B train.

04

Keep or merge
Kept separate, adapters swap per request on one frozen model. Merged into the weights, the model runs with no added latency.

LoRA in practice

WHERE IT IS USED

- Most open-model tuning services: Together, Fireworks, Tinker (LoRA only), SageMaker AI, and Hugging Face's PEFT (parameter-efficient fine-tuning) library. Basically all of them.
- OpenAI, Microsoft Foundry and AWS Bedrock expose no LoRA setting.

PROS

- Far less training memory, because only the adapter is trained.
- One frozen model serves many adapters, one per task or customer, and a bad one is easy to roll back.
- It forgets less than full fine-tuning.

CONS

- Learns less than full fine-tuning on large datasets, such as code and maths with about 100,000 examples.

TECHNIQUE	LEARNS FROM	NEEDS FROM YOU	COST	WHERE YOU CAN RUN IT	MAIN RISK
SFT	Example answers	About a hundred or more good examples	Low	Closed and open models	Imitates; weak for facts; forgetting
RL	Scores on its own attempts	An automatic check or reward	High	Few managed services; open models via training APIs	Reward hacking
RLHF	A reward model of preferences	Ranked answers, and compute for several models	Highest	Labs; rarely teams	Longer answers, agreement with the user
DPO	Preference pairs	Good pairs	Low to medium	Some closed models; open models	Offline; only as good as the pairs
Distillation	A teacher's outputs	Inputs, a teacher the terms allow, a test set	Low	Closed models (teacher and student from one family) and open models	Inherits the teacher's errors
LoRA	Any of the above	A choice of settings	Lowers the cost of the others	Most open-model services	Learns less on large datasets

What we can and cannot learn with post-training

Tuning changes some things reliably and others poorly, so the first question is what kind of failure you have.

TARGET	WHAT IT TAKES	HOW RELIABLE
Format, tone, persona, stopping	About 100 to 1,000 examples	High; gains level off early
Skills the model already has, such as a reasoning style or the shape of a tool call	About 1,000 curated examples to bring them out; 100,000 or more to keep improving	Rises with data quality and scale
New facts, by SFT on questions and answers	Every fact covered, in many phrasings	Slow; hallucination rises; below retrieval where compared
Facts that change weekly	Not a tuning target: use retrieval	Not reliable: a new training run for every change

When to change the weights: the order

Change weights only after the cheaper steps fail. Each earlier step costs less, changes faster and survives a model update.

